Open your terminal. Navigate to the **A4** folder with the **cd** command. Enter **dune build**.

To run the main program:

The main program allows you to compare two text files and print out a set containing all of the inputs the two textfiles have in common in order. There are two possible ways of comparison: Case sensitive and case insensitive comparison. First, put the textfiles you want to compare into the A4 folder, then:

To compare two textfiles in a case insensitive manner, enter:

dune exec bin/main.exe -- -i "name_of_first_textfile" "name_of_second_textfile"

Remember to put the names of the textfiles in double quotations. The output will be a set printed out in string format on the terminal that differentiates upper and lower case characters. For instance, if "Apple" and "apple" are in both textfiles, they would both be in the output printed set as they have different cases in one of their characters. For the output printed set, strings starting with numbers(in numeric form, e.g 0,1,2,3…) are at the front, followed by strings starting with upper case letters in alphabetical order(A,B,C…) followed by strings starting with lowercase letters in alphabetical order(a,b,c….).

To compare two textfiles in a case sensitive manner, enter:

dune exec bin/main.exe "name_of_first_textfile" "name_of_second_textfile"

Remember to put the names of the textfiles in double quotations. The output will be a set printed out in string format on the terminal that don't differentiate upper and lower case characters. For instance, if "Apple" and "apple" are in both textfiles, they would not both be in the output printed set as both strings have the same characters, so only one will be in the . For the output printed set, strings starting with numbers(in numeric form, e.g 0,1,2,3…) are at the front, followed by strings starting with letters in alphabetical order(A,b,C, d…) regardless of case.

To use utop:

Enter **dune utop**. Then, enter **#use "lib/sets.ml";;**.

To create a module that allows you to build case sensitive sets, enter:

module NameModule = BuildSet(SenSets);;

Where **BuildSet** is the functor and **SenSets** is the module it takes in. **NameModule** can be changed to any random module name you want.

To create a module that allows you to build case insensitive sets, enter:

module NameModule = BuildSet(InsenSets);;

Where **BuildSet** is the functor and **InsenSets** is the module it takes in. **NameModule** can be changed to any random module name you want.

1. To create an empty set, use the **empty** function. It requires no arguments. Type in:

let set_name = NameModule.empty;;

set_name can be changed to any random variable name you want.

2. To add an element to a set, use the **add** function. The **add** function needs two arguments: the element you want to add and the set you want to add the element two. The add argument will output a new set. Type in:

let set_name = NameModule.add element set_added;;

set_name can be changed to any random variable name you want. **element** is the element you want to add to the set, and **set_added** is the set you want the element to be added to. Ensure that your element has the same type as the elements of the set you are adding it to, or otherwise the function will not work. If your **element** is already present inside **set_added**, the set returned will not include it as a duplicate.

3. To turn a set into a string, use the **to_string** function. The **to_string** function needs one argument: the set you want to turn into a string. The **to_string** function will output the set you inputted as an argument as a string. Type in:

let outcome_string = NameModule.to_string turn_string;;

outcome_string can be changed to any random variable name you want. **turn_string** is the set you want to turn into a string, parsed in as an argument. **outcome_string** will be formatted like this if **turn_string** contains elements:

“{element_one,element_two,....element_n,}”

Where each element is followed by a comma, and the entire set is enclosed by {}.

If **turn_string** does not contain elements, the output will be “**{}”**.